

Using plate solves to calibrate the BNO055 IMU — assessment + implementation plan

Date: 2026-07-21 **Question:** Plate-solve results could calibrate the IMU while it is in use. The BNO055 has no flash to persist calibration across power cycles, so any calibration would have to be saved externally. Investigate and assess.

TL;DR

- The plate-solve → IMU calibration loop the question imagines **already exists and runs live**, in two forms: absolute attitude **anchoring** (`imu_ref`) and the camera ↔ IMU **extrinsic** fit (`imu_frame_R` , Kabsch). The real gap is **persistence** — not a missing calibration.
 - The highest-value thing to persist is the **plate-solve-derived extrinsic** (`imu_frame_R`), which is physically fixed but re-learned from scratch every power cycle. This is exactly the "no flash → save it" idea, pointed at the right target.
 - Persisting the **BNO055's own accel/gyro offset profile** (the classic no-flash workaround) is worth doing but **lower** value: in IMUPLUS mode only gyro/accel matter, the chip re-derives them anyway, and re-anchoring already bounds drift. Plate solves act as the *validator/gate*, not the source, for this profile.
 - **Do not** use plate solves to re-enable the magnetometer / NDOF — the interference is uncalibratable and plate solves already are the absolute heading reference.
-

Current state (as of v0.11.59)

1. The BNO055 runs in IMUPLUS mode — magnetometer disabled

`imu_proc.py` initialises the chip in `_OPR_IMUPLUS` (accel + gyro fusion, mag off) deliberately: the magnetometer is unreliable near the telescope's metal body and motor drives. Consequences:

- **Pitch/roll** are gravity-referenced → absolute, no drift.
- **Heading/yaw** has no absolute reference → drifts ~1–5°/hr from gyro bias.
- Only **accel + gyro** calibration is meaningful. Gyro calibrates by holding still a few seconds; accel from a few static orientations. **Mag calibration is moot.**

2. No BNO055 calibration registers are touched; nothing IMU-derived persists

`imu_proc.py` never reads the calib-status register (0x35) or the offset/radius blob (0x55–0x6A); the chip auto-calibrates internally each power-up. The only IMU config keys are `imu_rate_gate_dps` and `imu_exact_predict` (`config.py:265,272`). `imu_frame_R` , `imu_calib_C` , and `imu_ref` live only in `shared_cfg` and are **rebuilt from scratch every session**.

3. Plate solves already calibrate the IMU — two live mechanisms

- **Absolute attitude anchoring** (`imu_ref` , `solver_proc.py::_imu_update_reference` , ~line 809): every successful solve re-anchors the IMU dead-reckoning to the solved attitude. Drift never accumulates beyond one inter-solve interval. This is loosely-coupled plate-solve ↔ IMU fusion, already shipped.
- **Camera ↔ IMU extrinsic** (`imu_frame_R` , Kabsch fit in `imu_frame.py` ; legacy 2-D `imu_calib_C`): fit live from (`r_imu` , `r_sky`) rotation-vector pairs harvested between consecutive solves, quality-gated (≥ 4 magnitude-consistent pairs, axis diversity ≥ 0.25 , $R^2 \geq 0.9$). The mounting rotation M (IMU body → camera) satisfies $r_{sky} = R \cdot r_{imu}$; the exact-quaternion LX200 prediction (`comms_proc._imu_predict`) uses it. This is literally "plate solves calibrating the IMU mounting as it is used."

So the plate-solve → IMU calibration already runs. What it lacks is **memory**: the mounting fit and any sensor bias are discarded on every power cycle, and the extrinsic only becomes observable *after* the scope has slewed in ≥ 2 non-collinear directions with a good solve at each end — before that the hint path uses the wider-cone body-frame fallback.

Opportunities, ranked

Opportunity 1 — persist the plate-solve-derived extrinsic (`imu_frame_R`). Highest value.

The IMU → camera mounting M is physically fixed (changes only on remount) yet is re-learned every session and is unavailable until the first multi-direction slew converges. Persisting it makes the exact-quaternion prediction available **from the first solve after boot**.

- **What to save:** the 9-float `imu_frame_R`, its quality dict (`n`, `r2`, `axis_diversity`), and a timestamp.
- **Where:** `/var/lib/diofinder/imu_extrinsic.json`, using the same atomic write discipline as `seeing_overrides.json` / the hot-pixel mask.
- **Self-heal (load-bearing):** treat the loaded value as a **seed**, keep the live Kabsch fit running, and override the stored value when the live fit is good and diverges from it beyond a threshold (a remount must recover automatically). Same philosophy as the FOV drift-recommit.
- **Plate-solve role:** direct — this artifact *is* a plate-solve product.

Opportunity 2 — persist the BNO055 accel/gyro calibration profile. Medium value.

The classic no-flash workaround, and the literal thing the question raises. Once the chip reports gyro+accel calib status = 3, read the 22-byte blob in CONFIG mode, save it, and write it back at next boot before switching to IMUPLUS. diofinder's init already does a CONFIG → IMUPLUS excursion, so restore slots in cleanly *before* the mode switch.

- **Benefit:** skips the cold-start gyro-bias warm-up so dead-reckoning is trustworthy immediately after boot.
- **Honest caveats:** in IMUPLUS only gyro/accel offsets are used (mag offsets in the blob are unused); the chip re-zeroes gyro whenever the scope sits still (constantly, between slews); and re-anchoring already bounds drift. So this is a warm-up nicety, not a correctness fix.
- **Plate-solve role:** *validator/gate*, not source. Plate solves don't feed the chip's internal accel/gyro cal (that comes from gravity/motion), but the agreement between IMU deltas and solved deltas is the right signal for "this profile is good, save it."

Opportunity 3 — plate-solve gyro-bias feed-forward. Optional, defer.

Estimate a slowly-varying gyro-bias vector from the IMU-vs-solve delta mismatch per inter-solve interval and de-drift the between-solve prediction. Low marginal value: re-anchoring resets the reference every solve, so over a ~10 s interval even 5°/hr is only a few arcmin; it mainly helps **fast-slew** dead-reckoning latency, partly duplicates the chip's own gyro auto-cal, and can only be applied in the software layer (the BNO055 emits a fused quaternion, not debiasable raw gyro). Revisit only if slew-time pointing becomes a pain.

Anti-recommendation — do not re-enable the magnetometer / NDOF.

Mag is off because of hard-iron interference (scope metal, motors): a spatially-varying, non-constant field no calibration can fix. Plate solves already are the absolute heading reference, so NDOF would inject drift-prone noise for zero benefit.

Implementation plan

Two independent, additive units. Ship **Unit A first** (higher value, no hardware-register risk); **Unit B** is optional polish.

Unit A — persist + self-heal the camera ↔ IMU extrinsic

New module `diofinder/imu_persist.py` (pure, hardware-free, unit-testable):

```
# imu_persist.py
_PATH = "/var/lib/diofinder/imu_extrinsic.json"

def save_extrinsic(R9, quality, path=_PATH): ... # atomic temp-file + os.replace
def load_extrinsic(path=_PATH): ...           # -> (R9, quality, mtime) | None
def clear_extrinsic(path=_PATH): ...
def extrinsic_diverged(R_live, R_saved, tol_deg=2.0) -> bool:
    # relative rotation angle between the two 3x3 matrices; > tol => remount
```

Reuse the atomic-write helper already used for `seeing_overrides.json` (do not re-implement temp-file/ `os.replace`).

solver_proc.py — seed on boot, persist on good fit, self-heal on divergence:

1. **Boot seed.** Where the solver initialises IMU state, call `load_extrinsic()`; if present, publish `shared_cfg["imu_frame_R"] = R9` and mark it provisional (`imu_frame_quality = {"source": "persisted", ...}`) so the exact-prediction path in comms is live from the first solve.
2. **Persist on good fit.** In `_imu_update_reference`, when the live Kabsch fit produces a fresh `R9` whose quality clears a *stricter* save gate (e.g. `n >= 8`, `r2 >= 0.97`, `axis_diversity >= 0.4` — tighter than the *use* gate so only a well-observed mounting is written), and it differs from the last-saved value, call `save_extrinsic(R9, quality)`. Throttle writes (e.g. at most once per N solves or when quality improves) to avoid disk churn.
3. **Self-heal.** If a persisted seed is in force and a good live fit (`extrinsic_diverged(R_live, R_seed)` true) disagrees beyond `tol_deg`, adopt the live fit and overwrite the file — this recovers a remount without user action, mirroring the FOV drift-recommit.

Factory reset. Add `imu_extrinsic.json` to the artifacts `diofinder-factory-reset --clear-*` can delete (alongside overrides / hot-pixel mask).

Tests (`tests/test_imu_persist.py` , hardware-free): round-trip save/load; `extrinsic_diverged` true/false at known angles; atomic-write leaves no partial file on simulated failure; the stricter save gate rejects a marginal fit.

No comms change required — `_imu_predict` already consumes `imu_frame_R` transparently; it simply starts finding it populated at boot.

Unit B — persist + restore the BNO055 accel/gyro profile

`imu_proc.py` — register I/O around the existing CONFIG → IMUPLUS init:

1. **Add register constants:** `calib-status 0x35` , `calib blob 0x55 – 0x6A` (22 bytes: accel/mag/gyro offsets + accel/mag radii).
2. **Restore before the IMUPLUS switch.** In `_probe_and_init` , after entering `_OPR_CONFIG` and before writing `_OPR_IMUPLUS` , if `/var/lib/diofinder/bno055_calib.json` exists, block-write its 22 bytes to `0x55` . (Writing calib registers is only legal in CONFIG mode — the current sequence is already in CONFIG at that point.)
3. **Save when good, gated by plate solves.** In the poll loop, periodically read the `calib-status` byte; when **gyro and accel bits both read 3** and `shared_cfg` shows recent solves whose IMU deltas agree with solved deltas (reuse the `imu_frame_quality` / recent-solve signal — the plate-solve gate), read the 22-byte blob (requires a brief CONFIG excursion: switch to CONFIG, read, switch back to IMUPLUS — ~30 ms, do it at most once per session or when status first reaches 3) and write it via the same atomic JSON helper.
4. Expose `imu_calib_status` (sys/gyro/accel) in `shared_cfg` for the web UI / `version /status` surfaces, and a maint `imu_calib_clear` to delete the file.

Factory reset. Add `bno055_calib.json` to the clear list.

Risk notes for Unit B: the CONFIG excursion briefly stops fusion output — schedule it only when `imu_available` and the scope is idle (no active slew), and never during an align hold. A stale/temperature-drifted profile is still a valid *seed*; the chip re-converges, so a bad restore self-corrects. Keep Unit B behind a config flag (`imu_persist_bno055` , default off until field-validated).

Sequencing / rollout

1. Unit A behind no flag needed (pure software, self-healing) — but land it with the divergence self-heal from day one so a remount can't strand a user on a wrong stored mounting.
2. Unit B behind `imu_persist_bno055=false` until a couple of nights confirm the CONFIG excursion is non-disruptive and the restore actually shortens warm-up.
3. Defer Opportunity 3 (gyro-bias feed-forward) entirely.

What this buys

- **Cold start after boot:** exact-quaternion pointing prediction available from the first solve (Unit A) instead of after the first multi-direction slew; trustworthy dead-reckoning immediately (Unit B) instead of after gyro warm-up.
- **No new failure mode for the shipped path:** both units are seeds the live machinery already overrides; the live Kabsch fit and re-anchoring remain the source of truth.

Bottom line

The premise is right — plate solves calibrate the IMU, and no on-chip flash means saving externally — but the leverage is inverted from the obvious reading. The plate-solve → IMU calibration loop already runs live; the missing piece is that its most valuable product, the **plate-solve-derived camera → IMU extrinsic**, is thrown away every power cycle. Persisting *that* (with live self-heal) is the first move; persisting the BNO055's own accel/gyro offsets is worthwhile polish; gyro-bias feed-forward and any magnetometer re-enable are not worth doing.